



PEJU/AND/ADA/APP WEB IA

DESARROLLO DE APLICACIONES WEB CON IA

M.3011.002.083

Desarrollo Backend - Diseño de servicios en la nube

Juan Manuel Castillo Andrades



Cofinanciado por
la Unión Europea



Fondos Europeos



Sistema Nacional de
Garantía Juvenil





API's

Un poco de contexto



El dispositivo que precipitó el cambio



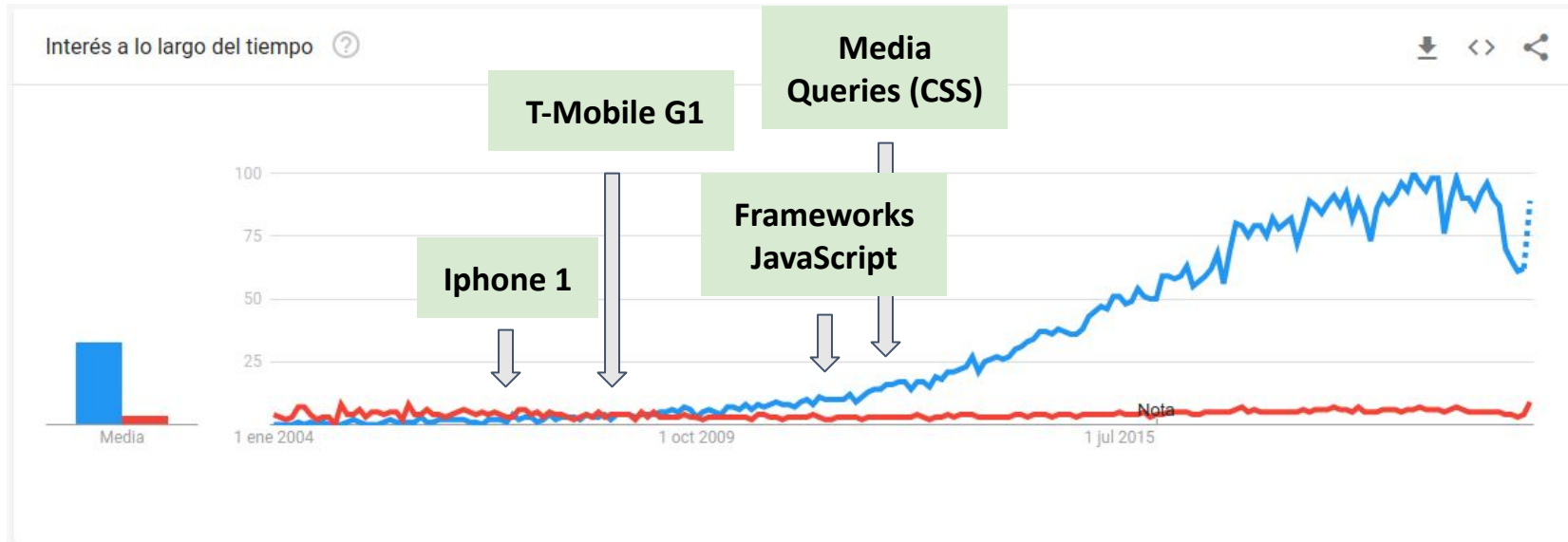
¿Cómo llegábamos a nuestros usuarios?

→ Navegador en un ordenador





Hitos que ayudaron al auge de las API's modernas



Evolución 2004-Actualidad búsquedas API REST (azul) frente a API SOAP (rojo)

Fuente: <https://trends.google.es/trends/explore?date=all&q=API%20REST,API%20SOAP>



¿Cómo llegamos a nuestros usuarios?

- Navegadores (móvil / pc)
- Redes sociales
- Video consolas
- Apps
- Smart TVs
- Relojes inteligentes
- Totems interactivos



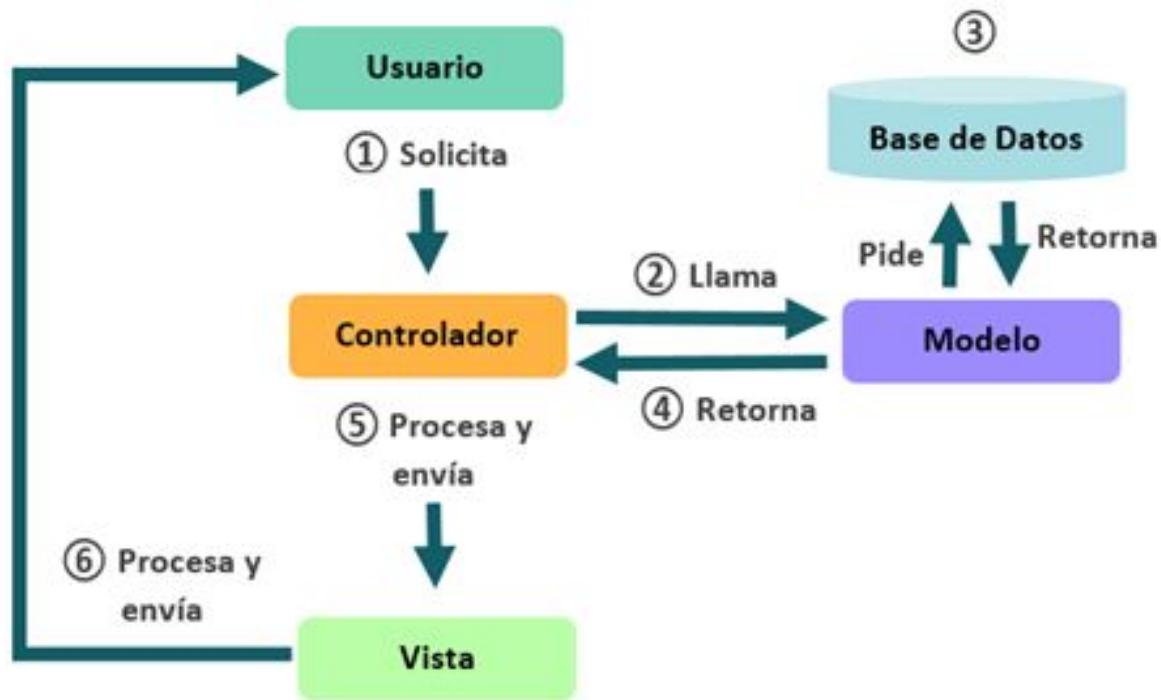


API's

Cambio de paradigma



Arquitectura acoplada - MVC



El monolito



El monolito supremo



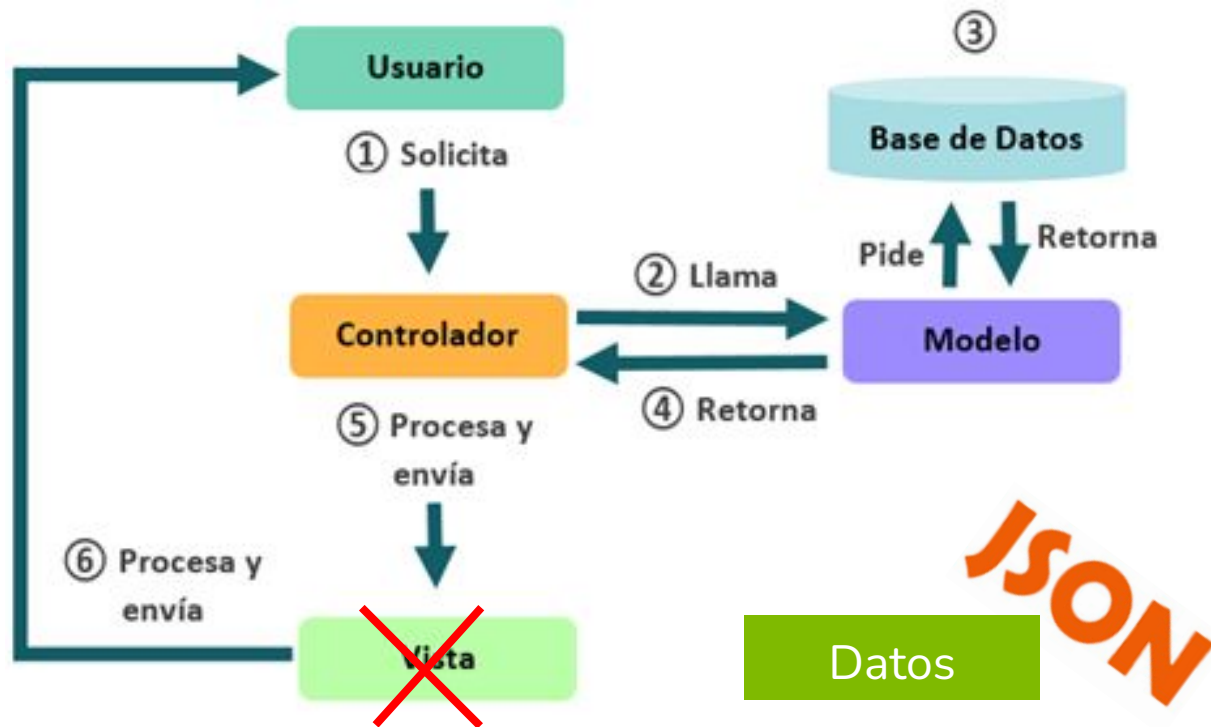


Arquitectura
desacoplada
(headless)



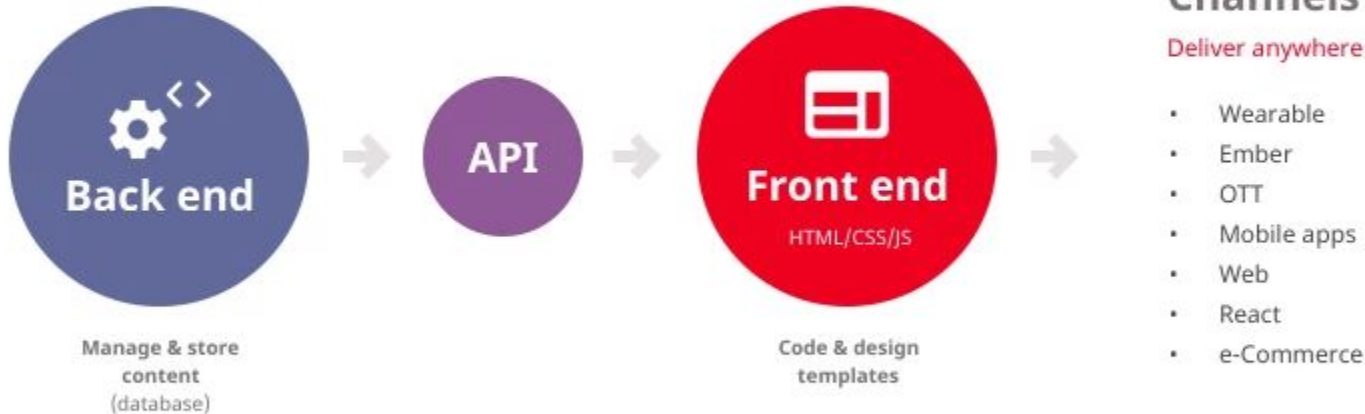


Arquitectura desacoplada/descabezada (headless)



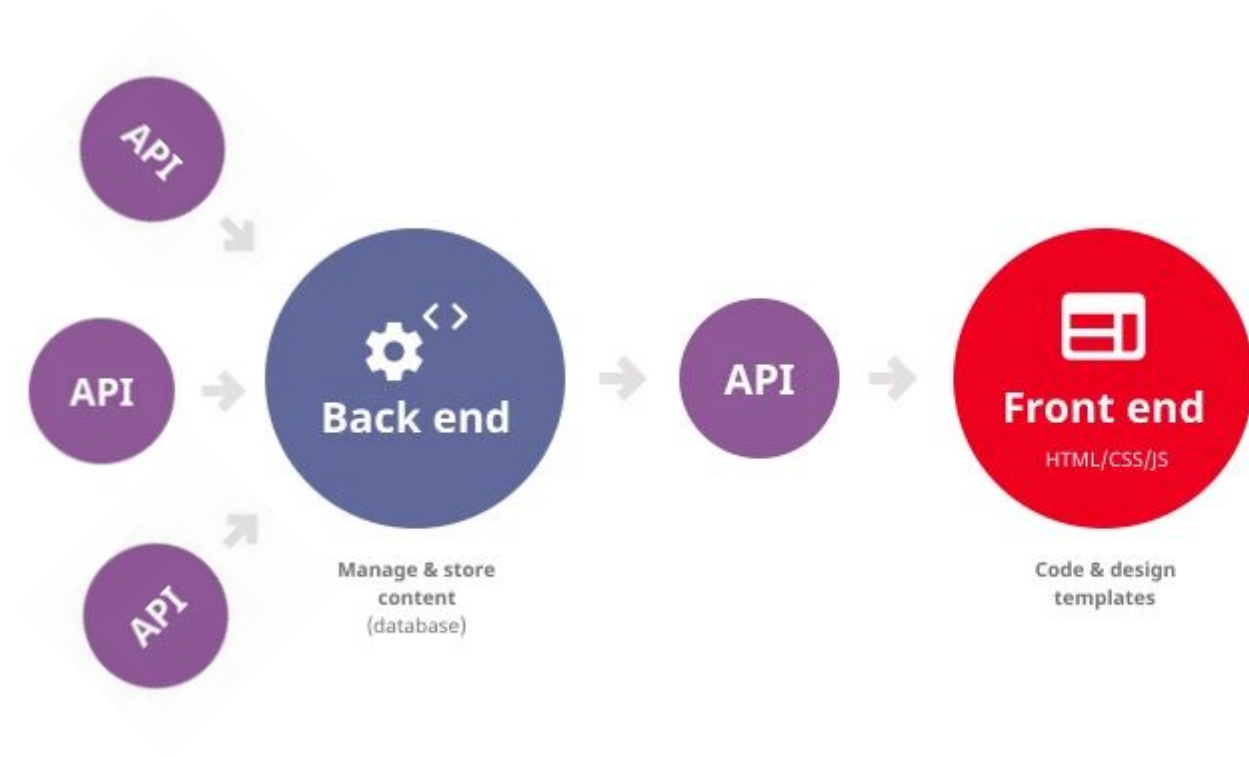


API REST - Arquitectura desacoplada (headless)



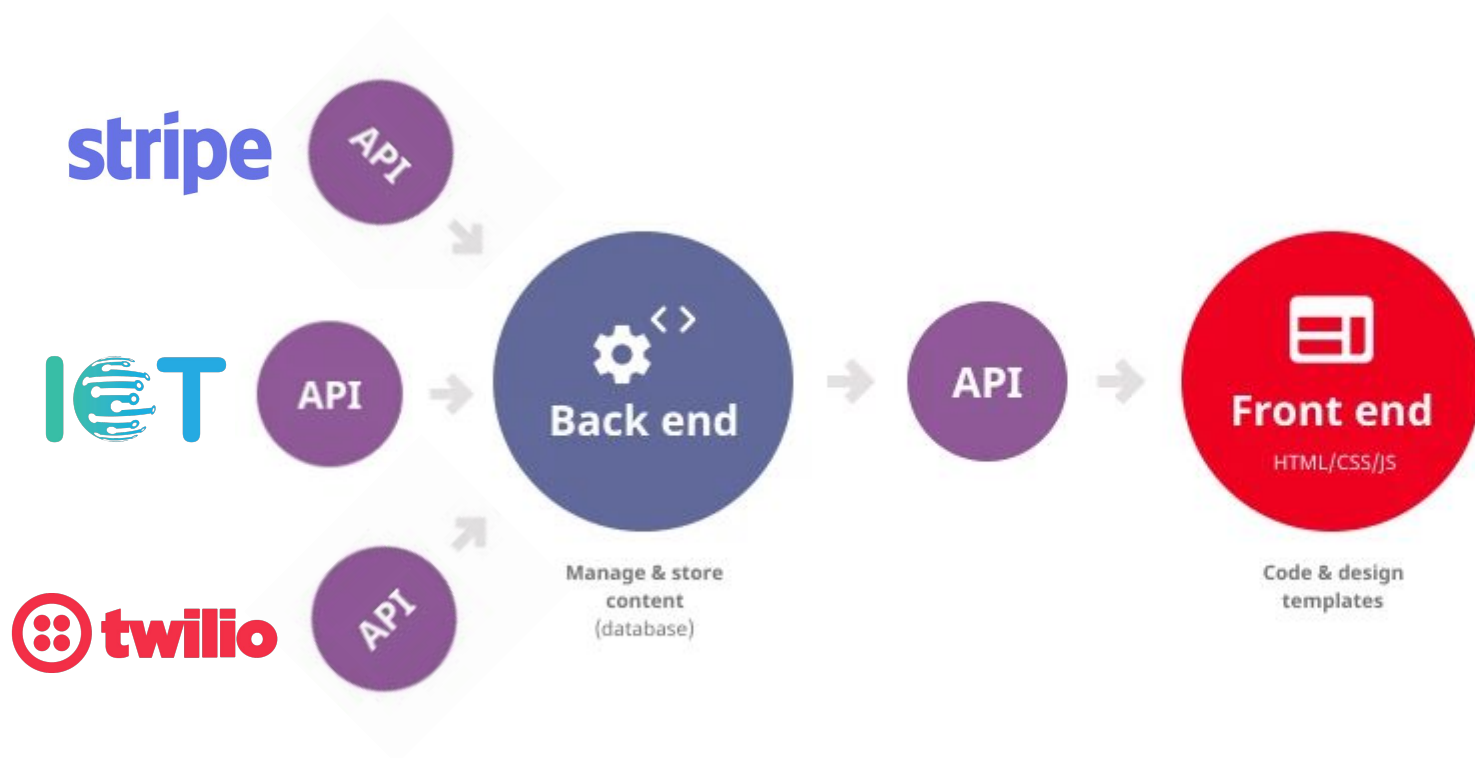


API REST - Arquitectura desacoplada (headless)





API REST - Arquitectura desacoplada (headless)





API's

¿Qué es?



¿Qué es una API?

***API** procede del inglés de **Application Program Interface**. Es un **conjunto de funciones y procedimientos** diseñados **para ser utilizados por otro software**.*

*Podemos decir que es un **acuerdo de comunicación** entre dos partes.*



API's Tipos



SOAP (Webservices)

- Es **complejo y difícil de entender**:
 - **XML** es muy verbose y poco claro
 - **WSDL** es complejo y la información muy extensa lo cual acopla mucho al consumidor
- **Mayor latencia** debido a la transferencia de información basados en XML por lo tanto no funciona bien en App móviles y RIA (Rich Internet Application)
- Usa **HTTP sólo como medio de transporte** por lo que no aprovecha:
 - Sistemas de **cacheado**
 - Negociación de **formatos**
 - Negociación del **idioma**
 - **Seguridad**
 - **No es firewall friendly** (aunque lo hayan vendido como tal)
- El **personal técnico requiere de mucha formación**
- **Consecuencia: Alto acoplamiento**

REST

- Es **simple y fácil de entender**:
 - **JSON** es claro y conciso
 - Se publican **recursos** por **grupos de rutas**
 - Cada **recurso** está **identificado de forma única**
- **Menor latencia** debido a la menor transferencia de información basados en JSON por lo tanto funciona muy bien en App móviles y RIA (Rich Internet Application)
- Usa **intensamente el protocolo HTTP** por lo que aprovecha:
 - Sistemas de **cacheado**
 - Negociación de **formatos**
 - Negociación del **idioma**
 - **Seguridad**
 - **Firewall friendly**
 - **Compresión**
- El **personal técnico requiere de poca formación**
- **Consecuencia: Bajo acoplamiento**



SOAP (Webservices)

419 bytes

```
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:user="http://example.com/user">
  <soapenv:Header/>
  <soapenv:Body>
    <user:UserDetails>
      <user:Nombre>Federico</user:Nombre>
      <user:Apellido>Clarín</user:Apellido>
      <user:Email>fedegmail.es</user:Email>
      <user:Edad>24</user:Edad>
    </user:UserDetails>
  </soapenv:Body>
</soapenv:Envelope>
```

REST

123 bytes (-241%)

```
{
  "user": {
    "nombre": "Federico",
    "apellido": "Clarín",
    "email": "fedegmail.es",
    "edad": 24
  }
}
```

{ REST }



¿Qué es una API REST?

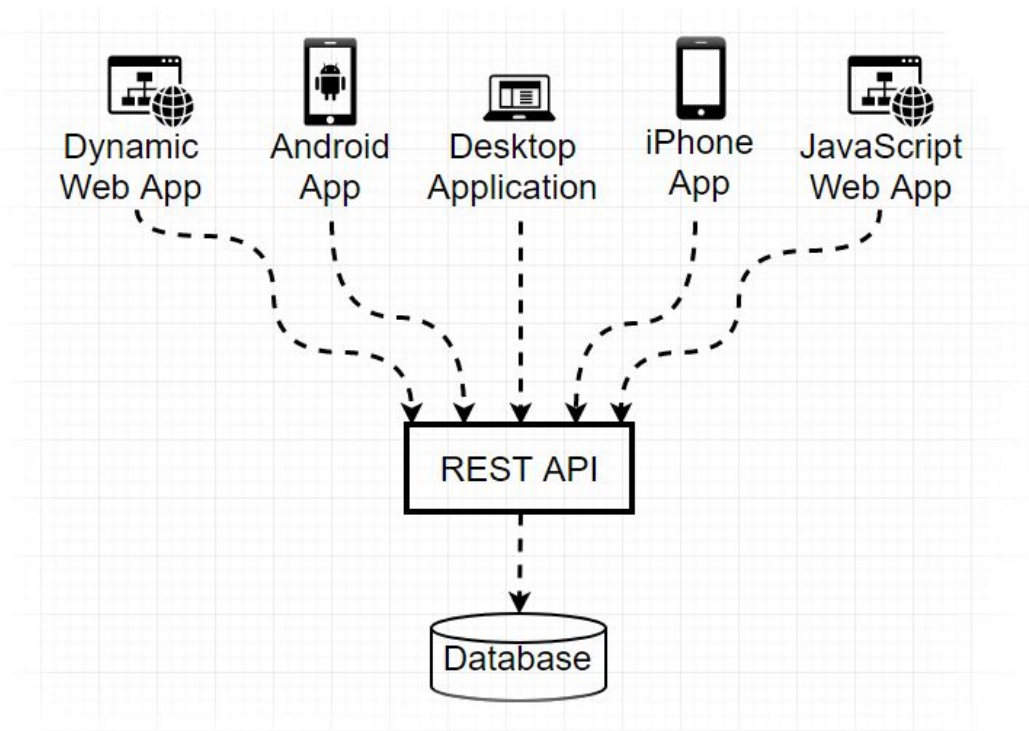
*“**RE**presentational **S**tate **T**ransfer es un tipo de arquitectura de desarrollo que **usa intensamente** en el protocolo HTTP”*

“REST puede ser considerado como un framework para construir interfaces de comunicación que respetan el protocolo HTTP”



¿Qué nos permite hacer una API?

Nos permite **comunicar** **nuestros datos** con tantos **consumidores** como queramos. Permite que éstos puedan ser de naturaleza muy diferente y usar tecnologías y lenguajes completamente distintos.





API REST - Características

Creado en el año 2000 por Roy Fielding (padre del protocolo HTTP).

- Cliente/Servidor **Sin estado** (una petición debe recibir toda la información que necesita para ser ejecutada)
- **Operaciones** basadas en los **verbos HTTP**: GET, POST, PUT, DELETE...
- **Manipulación** siempre **a través** de la URLs (llamados **endpoints**)

```
{protocol}://{domain/hostname}[:port]/{resource path}?{filters}
```

```
https://miapi.com/songs?year=2018&order=-year,-pubDate
```



API REST - Ventajas

- **Separa** cliente / servidor (frontend / backend)
- Fiabilidad y **escalabilidad**
- **Independencia** de tecnologías y lenguajes (cliente)
- Necesidad de **menos recursos** para funcionar (servidor)





API's

Casos de éxito



Caso de éxito - Tabla de mareas

Integra datos de diferentes API's (presión atmosférica, calendarios lunares, oleaje, mareas, predicciones meteorológicas...) y las analiza para mostrar información aplicada a la pesca cuyo mayor exponente reside el saber cual es el mejor día y hora para pescar.

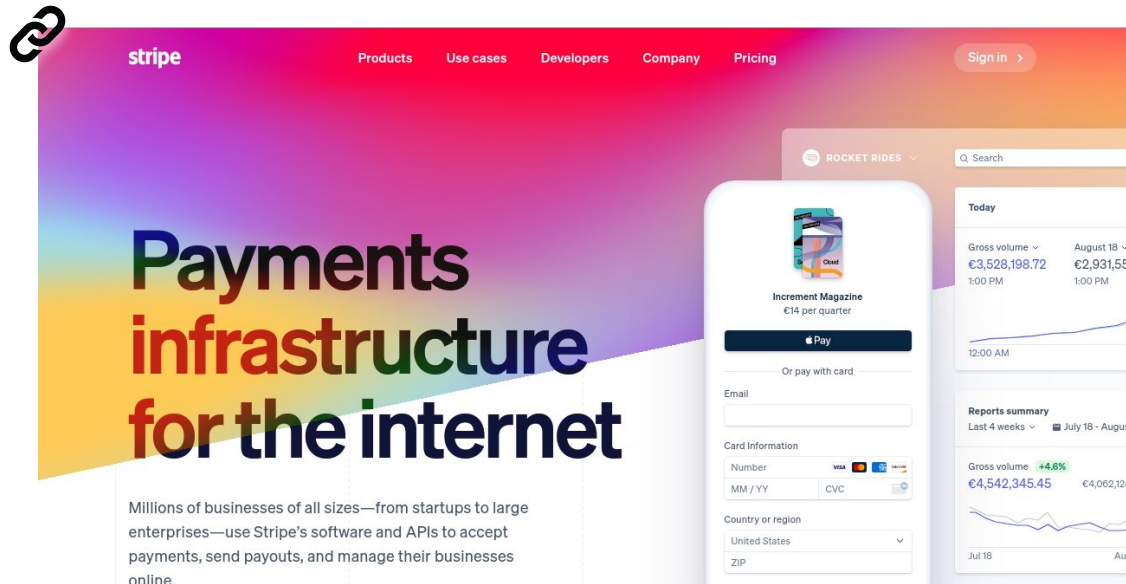
The screenshot displays the 'TABLADEMAREAS' web application interface for Huelva. The interface is divided into several sections:

- Header:** Includes a search bar with the location 'Europa > España > Huelva > Huelva', a settings gear icon, and a calendar for February 2021. The current time is 18:37 UTC+1, and the date is Tuesday, February 2nd, 2021.
- Left Sidebar:** Contains navigation links for 'Tiempo' (El tiempo, Temperatura del agua, Oleaje), 'Mareas' (Pleamares y bajamares, Coeficiente de mareas, Tabla de mareas), and 'Solunar' (Salida y puesta de la luna, Periodos solunares, Fase lunar, Observación astronómica).
- Main Content Area:**
 - TABLA DE MAREAS Y SOLUNARES:** A section titled 'Huelva' with a link to download the 'app de tabla de mareas'.
 - PREDICCIÓN • PESCA:** A large section with a background image of two people fishing. It includes the text 'Disfruta al máximo de una jornada de pesca en Huelva bien planificada' and a table with four colored buttons: 'Tiempo' (grey), 'Mareas' (blue), 'Solunar' (yellow), and 'Lugares' (pink).
 - tiempo:** A section titled 'EL TIEMPO HUELVA' showing the date 'HOY, MARTES, 2 DE FEBRERO DE 2021'.



Caso de éxito - Stripe

Su negocio tiene como corazón una API REST (SaaS) para la automatización de pago (puntuales o suscripciones). Integra pagos mediante tarjetas de crédito (Visa, Mastercard, American Express entre otros) independientemente del banco con el que trabajes.



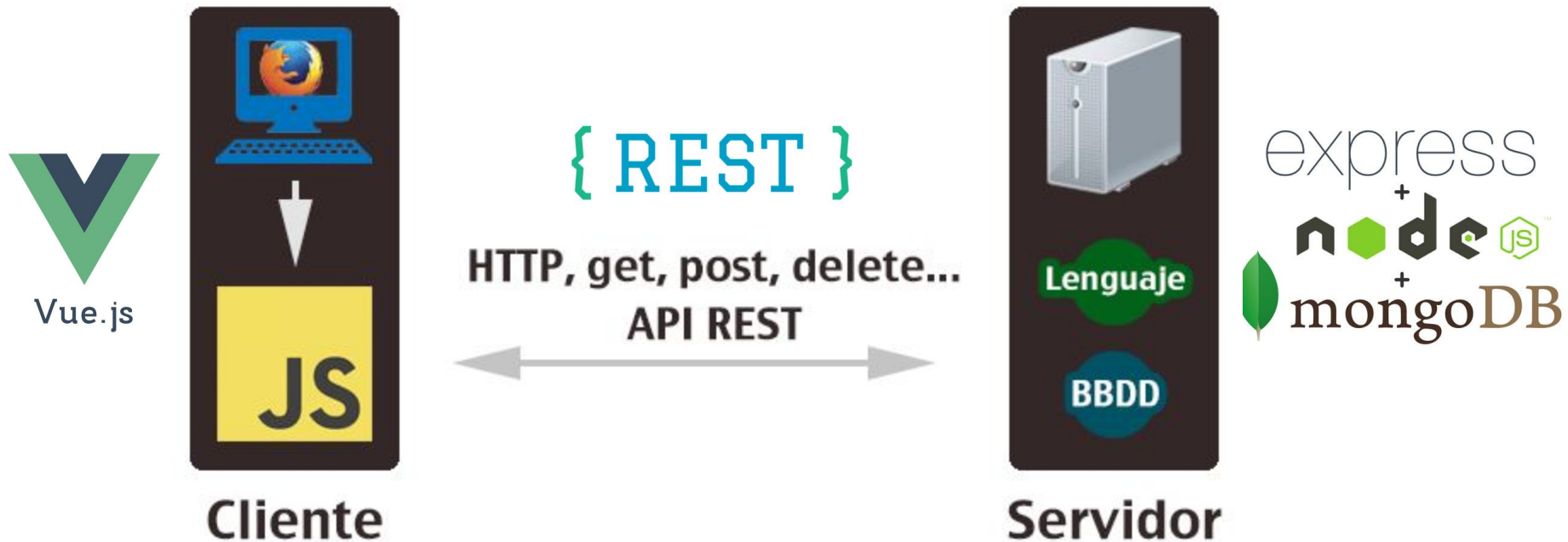


API's

¿Cómo encaja en nuestro Stack?



API REST - Nuestro stack tecnológico





API's

Diseñando nuestra propia API

ONE DOES NOT SIMPLY

CREATE AN API



API REST - Proceso de diseño

Una API (Acuerdo de comunicación) puede diseñarse mediante los siguientes pasos:

- Identificar **recursos**
- Definir la **información**
- Definir **rutas** y **operaciones**
- Definir **datos petición y respuestas**



API REST - Proceso de diseño

→ 1. Identificar recursos

Las API's exponen servicios relacionados con conjuntos de datos del negocio, es por ello que en primer lugar necesitamos identificarlos. Con este punto, podemos planificar los grupos de rutas a crear y profundizar en los servicios e información que contendrá nuestra API.

→ 2. Definir la información

Una vez identificados los recursos, podemos profundizar en la información que contendrá, es decir, detallar los campos que formarán cada uno de éstos. Es recomendable especificar al menos el nombre, tipo y si es obligatorio u opcional.



API REST - Proceso de diseño

→ 3. Definir rutas y operaciones

Al tener los recursos detallados con los campos que contendrán, estamos en disposición de definir los grupos de rutas para cada recurso, las operaciones (verbos) que habilitaremos y quién puede solicitarlo.

Si tendremos niveles de permisos, es necesario definir justo antes de las rutas los diferentes niveles.

→ 4. Datos de petición y respuesta

Por último, con las rutas y verbos definidos, sólo nos quedaría definir la información que viaja en cada petición y respuesta (por ruta/verbo).

Es importante que definamos en las respuestas qué códigos pueden ser devueltos y en qué situación se da.



API REST - Resultado del proceso de diseño

Un **documento técnico muy valioso** del que podemos extraer:

- Recursos existentes
- Datos almacenados de cada recurso
- Rutas y verbos disponibles por recurso
- Detalle de las peticiones y respuestas esperadas (por ruta/operación)
- Jerarquía de relaciones (si se pueden extraer de las rutas)



API REST - Resultado del proceso de diseño

Un **documento técnico muy valioso** del que podemos extraer:

- Recursos existentes
- Datos almacenados de cada recurso
- Rutas y verbos disponibles por recurso
- Detalle de las peticiones y respuestas esperadas (por ruta/operación)
- Jerarquía de relaciones (si se pueden extraer de las rutas)



API REST - Tips para el diseño de una API

- **Simplicidad:** Debe ser fácil de usar pero mantener la tolerancia al cambio (**equilibrio entre flexibilidad-usabilidad**). Hacernos preguntas sobre cómo de fácil es realizar ciertas peticiones o simular procesos del negocio pueden darnos una pista de nuestro diseño.
- **Declarativa:** Las rutas e información requerida para cada operación debe ser fácil de entender tanto en lo que hace como en lo que podemos esperar como resultado. Que sea consumida por otro software no implica que deba ser difícil de entender para un humano.
- **Consistencia:** El comportamiento de nuestra API y la experiencia de usuario al realizar operaciones con ella debe ser similar a otras existentes que realicen u ofrezcan servicios parecidos.
- **Tolerancia al fallo:** Cómo de permisiva es nuestra API frente a olvidos y errores por parte del usuario.
- **Estándares:** Debemos preguntarnos si se usan estándares.

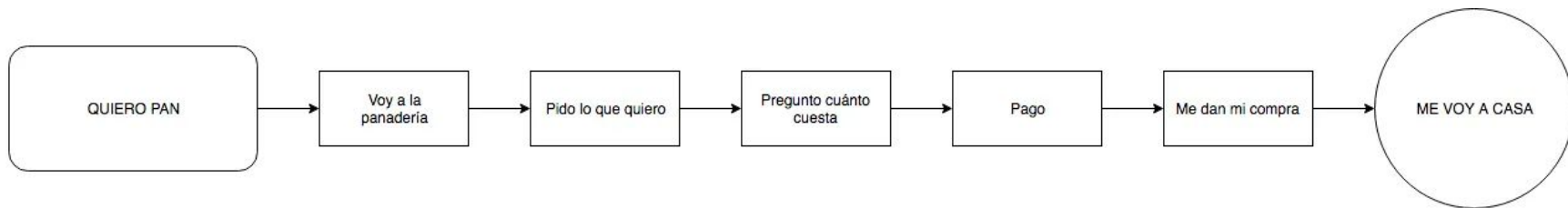


Diseño de API's

Huyendo del “happy path”



Happy Path

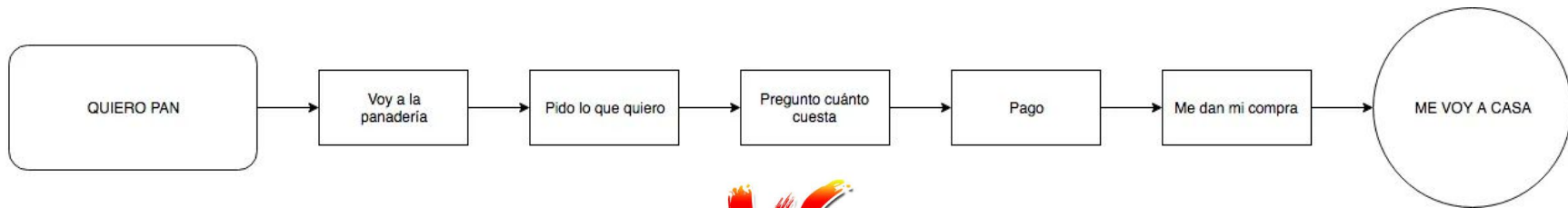




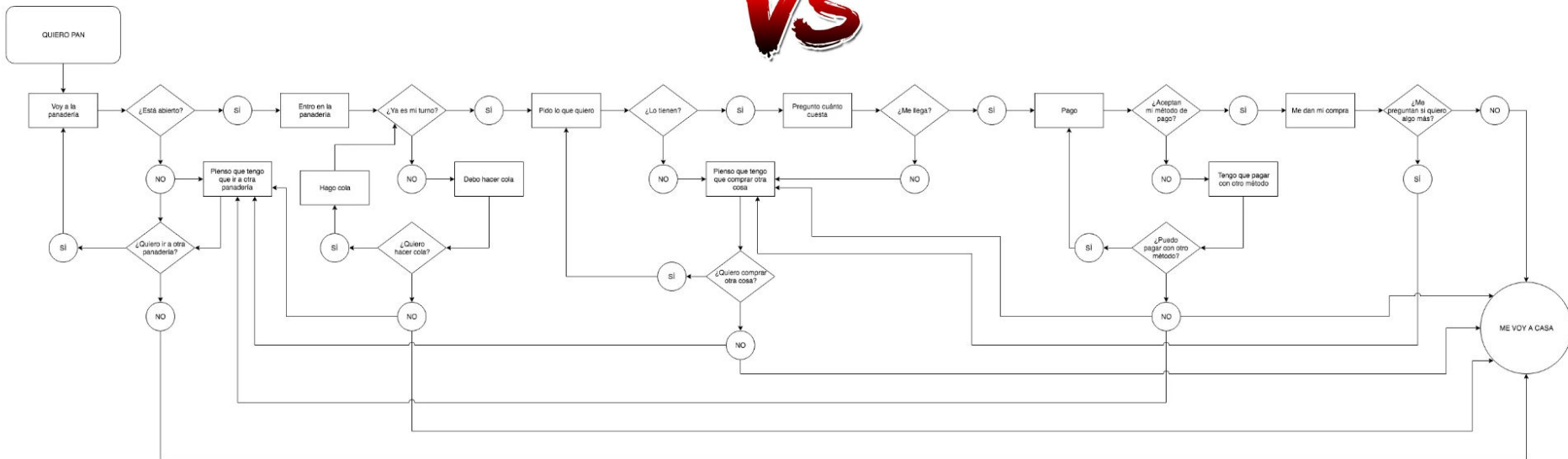
Happy

Path





VS





Diseño de API's

Niveles de implementación



API REST - Proceso de diseño

Según [el modelo de Richardson de madurez en las API's](#), varía entre los siguientes niveles según su implementación:

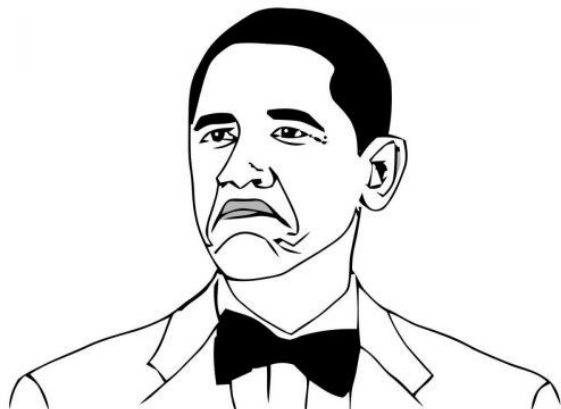
- 0. Uso del **protocolo HTTP**
- 1. Uso correcto de **rutas**
- 2. Uso correcto de **verbos y códigos de estado**
- 3. Implementación **HATEOAS**



API REST - Niveles de calidad

Existen **tres niveles de calidad** a la hora de diseñar una API REST

→ Uso correcto de **URL's**





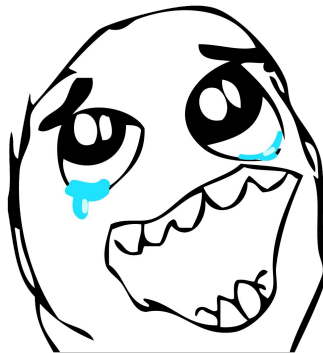
API REST - Niveles de calidad

Existen **tres niveles de calidad** a la hora de diseñar una API REST

→ Uso correcto de **URL's**



→ Uso correcto de **verbos y códigos de estado**





API REST - Niveles de calidad

Existen **tres niveles de calidad** a la hora de diseñar una API REST

→ Uso correcto de **URL's**



→ Uso correcto de **verbos y códigos de estado**



→ Implementación de **HATEOAS**





¡PONGÁMONOS MANOS A LA OBRA!



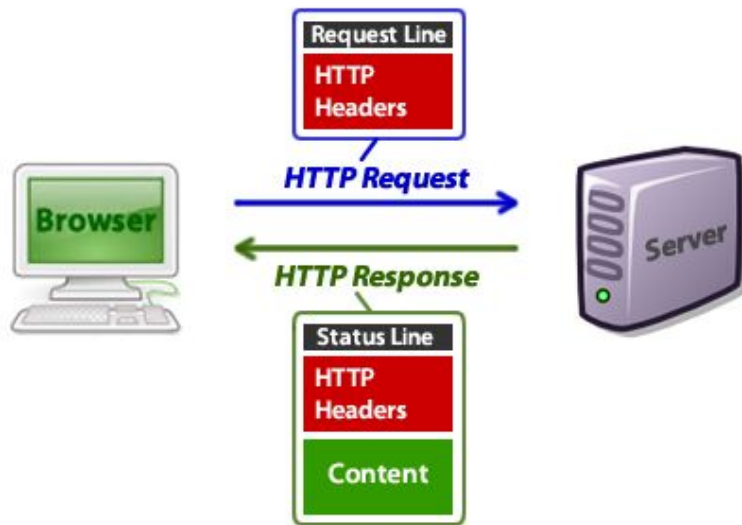
API REST

Uso del protocolo HTTP



API REST - Uso del protocolo HTTP

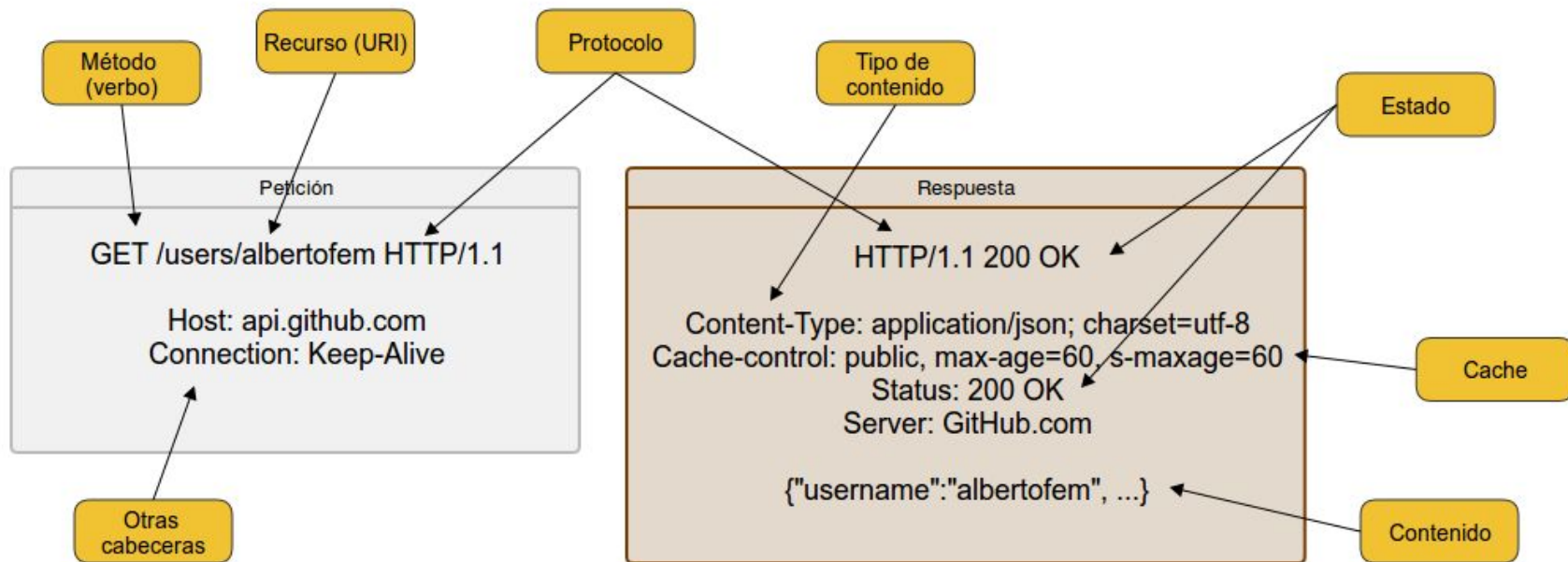
- Con cada recurso que se necesite (datos, imágenes, documentos...) es necesario enviar una petición la cual tendrá asociada una respuesta.





API REST - Uso del protocolo HTTP

→ Ejemplo de petición / respuesta





API REST - Uso del protocolo HTTP

→ Indicar el formato del recurso devuelto según los tipos aceptados.

Request

=====

GET /books/11

Accept: application/epub+zip , application/pdf, application/msword

← Solicitud del recurso en alguno de estos formatos por orden de preferencia

Response

=====

Status Code 200

Content-Type: application/pdf

← indica el tipo de recurso devuelto



API REST - Uso del protocolo HTTP

→ Indicar el formato de los datos enviados.

Request

=====

POST /books/11/ratings

Content-Type: application/vnd.rating.thumbsup



Envía la valoración con el formato dedo arriba o dedo abajo

Request

=====

POST /books/11/ratings

Content-Type: application/vnd.rating.stars



Envía la valoración con el formato 0-5 estrellas

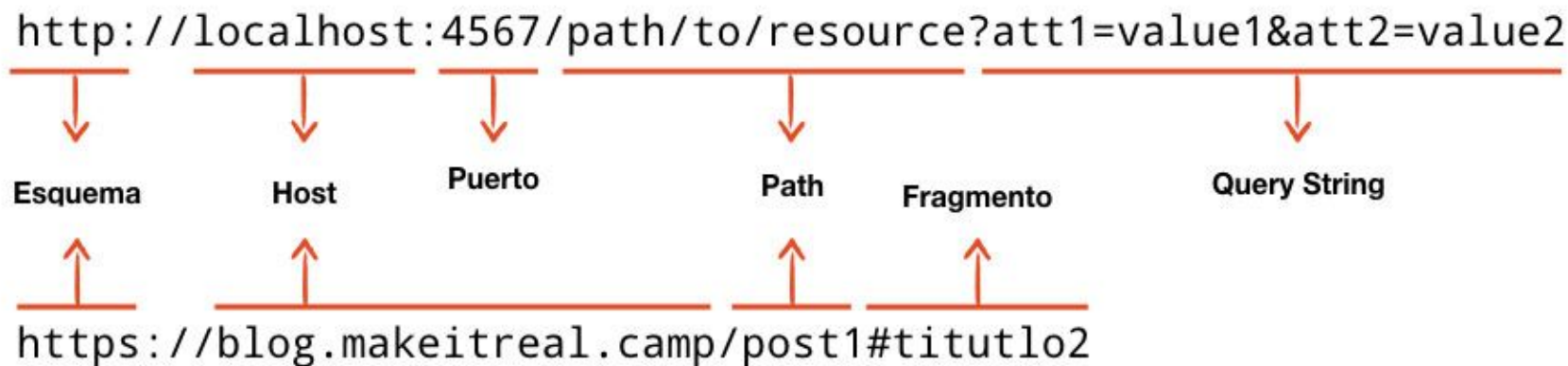


API REST

Uso correcto de URL's



API REST - Partes de una ruta





API REST - Uso correcto de URL's



→ **No deben indicar acción**, usa nombres y no verbos

```
[GET] /films/new
```

```
[POST] /films
```



API REST - Uso correcto de URL's



→ Deben ser **únicas** por lo que no debemos tener distintas url's para un mismo recurso.

```
[GET] /category/29/films/566
```

```
[GET] /films/566
```



API REST - Uso correcto de URL's



→ Deben mantener una **jerarquía con lógica**.

```
[GET] /invoices/47/clients/89
```

```
[GET] /clients/89/invoices/47
```

Preguntas: ¿Los usuarios tienen facturas o las facturas tienen usuarios?, ¿Los usuarios pueden existir sin facturas?, ¿Las facturas pueden existir sin usuario?



API REST - Uso correcto de URL's



→ Las operaciones de filtrado, ordenación y paginaciones no deben formar parte de la URL.

```
[GET] /films/year/2017/page/1
```

```
[GET] /films?year=2017&page=1
```

```
[GET] /films?year=2017&page=2
```

```
[GET] /films?year=2018&page=1&sort=-year
```



API REST - Uso correcto de URL's



Otros consejos

- ➔ Usa el **plural** en lugar del singular
- ➔ URL's **cortas** (no más de 3 nodos de jerarquía)
- ➔ Evitar guiones bajos y medios
- ➔ Implementa el **filtrado, ordenación y paginación** en colecciones
- ➔ Las rutas no deben cambiar con el paso del tiempo



API REST

Uso correcto de códigos de estado y verbos



API REST - Uso correcto de códigos de estado y verbos



→ Hacer uso de los métodos de petición HTTP (llamados verbos) adecuados para cada acción.

```
[GET] /films //obtiene todas las películas
[GET] /films/11 //obtiene una película por su id
[POST] /films //crea una nueva película
[PUT] /films/11 //actualiza una película por su id
[DELETE] /films/11 //borra una película por su id

[PATCH] /films/11 //actualiza parcialmente una película por su id
```



API REST - Uso correcto de códigos de estado y verbos



Create **Read** **Update** **Delete**



API REST - Uso correcto de códigos de estado y verbos



→ Hacer uso de los [códigos HTTP](#) adecuados para cada respuesta. [Cheat sheet](#)

```
[200] petición correcta
[201] nuevo recurso creado
[202] petición aceptada pero en proceso
[400] Petición incorrecta
[401] Petición no autorizada (requiere autenticación)
[403] Prohibido (autenticado pero sin permisos suficientes)
[404] Recurso no encontrado
[409] Conflicto con el recurso en el estado actual
```



API REST

Implementación HATEOAS



API REST - Implementación HATEOAS



*“**HATEOAS** procede de **Hypermedia As The Engine Of Application State**”*

“Viene a ser lo mismo que hipermedia como motor del estado de la aplicación”



API REST - Implementación HATEOAS



No os preocupéis, son “sólo” enlaces

```
{  
  "name": "Tatooine",  
  "rotation_period": "23",  
  "orbital_period": "304",  
  "diameter": "10465",  
  "population": "200000",  
  "residents": [  
    "http://swapi.co/api/people/1",  
    "http://swapi.co/api/people/2",  
    "http://swapi.co/api/people/4",  
    "http://swapi.co/api/people/6",  
    "http://swapi.co/api/people/10",  
    "http://swapi.co/api/people/12"  
  ]  
}
```



Enlaces de hipermedia



API REST - Implementación HATEOAS



*Nos permite **conectar mediante vínculos** las aplicaciones clientes con las APIs, permitiendo que estos clientes se olviden por conocer previamente cómo se acceden a los recursos.*

API REST - Implementación HATEOAS



```
{  
  "account": {  
    "account_number": 12345,  
    "balance": {  
      "currency": "usd",  
      "value": 100.00  
    },  
    "links": {  
      "deposit": "/accounts/12345/deposit",  
      "withdraw": "/accounts/12345/withdraw",  
      "transfer": "/accounts/12345/transfer",  
      "close": "/accounts/12345/close"  
    }  
  }  
}
```

En el siguiente ejemplo se exponen las acciones vinculadas a una cuenta bancaria.

El consumidor (un cajero) no tiene porqué conocer las rutas y puede ofrecerlas dinámicamente según sean devueltas por la API en función del estado del saldo bancario.



Operaciones permitidas sobre la cuenta

API REST - Implementación HATEOAS

```
{  
  "account": {  
    "account_number": 12345,  
    "balance": {  
      "currency": "usd",  
      "value": -25.00  
    },  
    "links": {  
      "deposit": "/accounts/12345/deposit"  
    }  
  }  
}
```

En un estado diferente, la API sólo devolverá los enlaces permitidos para la cuenta anteriormente consultada.



Con un saldo negativo, sólo ofrece el enlace para depositar



API REST - Implementación HATEOAS



HATEOAS nos permite conectar mediante vínculos toda la información contenida en la API. Los consumidores de éstas no tienen porqué conocer previamente cómo se acceden a los recursos y cuales existen, **pueden descubrirla de forma automática**.

Las restricciones impuestas por HATEOAS **desacoplan al cliente de nuestra API**. Esto permite a la funcionalidad del servidor evolucionar independientemente.

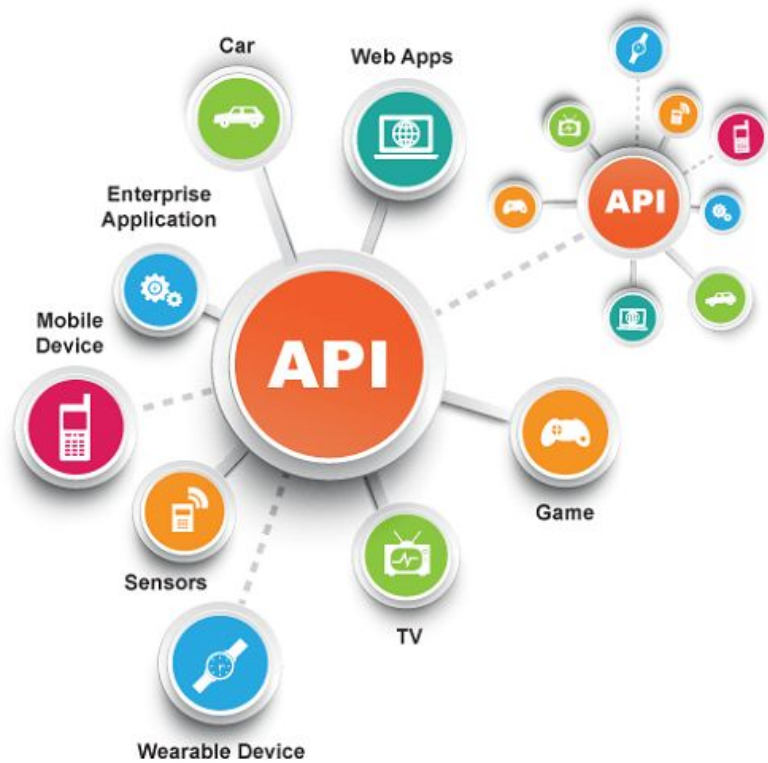
Es interesante su uso cuando:

- El cliente que consume la API no deba conocer o sepa reconocer las URL's
- Evitar mantenimiento en caso de que las URL's cambien (esto no debería ocurrir)
- Automatizar procesos entre API's sin que haya interacción humana.



API REST - Documentación

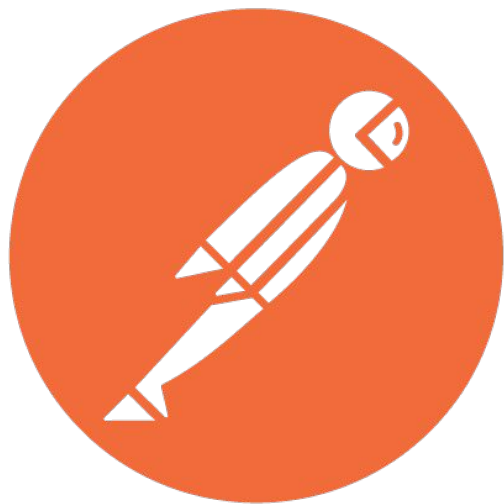
- [Cabeceras HTTP](#)
- [MIME Types](#)
- [Lista principales MIME](#)
- [Métodos HTTP](#)
- [Códigos de respuesta HTTP](#)
- [Negociación en CORS](#)
- [Estándar JSON API](#)





Consumiendo API's

Herramientas de trabajo

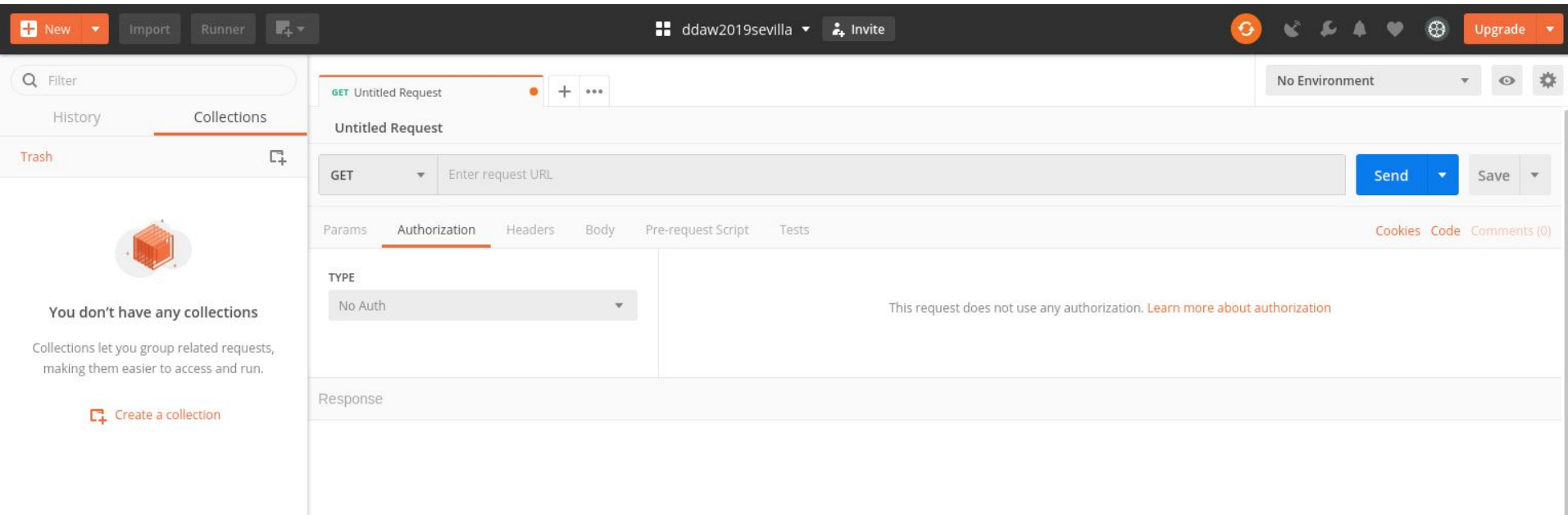


POSTMAN



API REST - POSTMAN

Aunque tiene muchas funcionalidades, el uso que haremos de POSTMAN será como cliente para nuestras API's.





Hoppscotch



Bruno

Re-inventing the API client
with Git



API's - Recursos

Recursos para integración y desarrollo de API's:

- API's públicas [aquí](#)
- [Devolviendo ficheros](#) en una API en Node + express
- Manejando [subida de ficheros](#) con nuestra API en Node + express
- [Dependencia](#) para gestión de subida de ficheros
- [Swagger](#) para la creación de documentación online

Práctica. Diseñar sobre papel los servicios para Pizza Delicious

Instrucciones

Tenemos que diseñar la API para nuestra app en Vue de Pizza Delicious. Nos debe permitir gestionar el catálogo de productos (categorizable), registro e identificación de usuarios (perfiles compradores y administradores) y el registro de pedidos con un control básico del estado de entregas así como recibir formularios de contacto.

1- Identificar recursos

2.- Crear un documento que nos sirva de guía para la fase de mocking y desarrollo posterior





Práctica. Mock Pizza Delicious

Instrucciones

Con los servicios ya diseñados sobre el papel para nuestra app en Vue de Pizza Delicious. Desarrollar en un proyecto en express con lo siguiente:

- 1.- Mock de los diferentes recursos
- 2.- Realizar llamadas al mock con un cliente HTTP (POSTMAN por ejemplo)
- 3.- Desarrollar la API con Node + Express

Notas

- 1.- No validar los datos recibidos inicialmente
- 2.- Añadir filtro **sort** para ordenación asc/desc en el listado de productos y pedidos (tomar como referencia el filtrado según el estándar JSON API)
- 2.- Utilizar faker (opcional) para generar 5-10 productos